

Data Structures & Algorithms

Complete Master Exam Notes, Definitions, Hinglish Explanations & Solved Examples

- COMPREHENSIVE GUIDE
- EXAM READY
- HINGLISH CLARIFICATIONS
- C CODE INCLUDED

1. Data Structure & Algorithm Fundamentals

Data Structure (DS) is a specialized format for organizing, processing, retrieving, and storing data in computer memory so that operations can be performed efficiently. It defines the algebraic structure of data, relationships among data elements, and the operations that can be applied.



EASY HINGLISH EXPLANATION:

Data Structure bas ek aisa tareeka (way) hai jisse hum computer ke inside data ko proper tarike se arrange aur organize karte hain, taaki jab bhi zaroorat ho, hum dynamic ya static memory se efficient way me data store, modify ya search kar sakein. Jabki DBMS database files handle karta hai, DS memory (RAM) level par operational efficiency pe focus karta hai.

Important Definition Table (Exam Hotspots)

Term / Concept	Technical Definition	Exam Memory Key
Abstract Data Type (ADT)	A mathematical model for data types where the data type is defined by its behavior (operations) from the point of view of a user, without specifying the internal implementation details.	"What it does, not How it is done" (e.g., Stack ADT defined by push/pop).
Primitive DS	Basic data structures directly operated upon by machine instructions (e.g., int, float, char, pointer).	Single value storage, fixed pre-defined memory size.
Non-Primitive DS	More complex data structures derived from primitive data types to store collections of data values (e.g., Arrays, Linked Lists, Trees).	Multiple values storage, built using pointers & dynamic allocation.
Linear vs Non-Linear DS	In Linear DS , elements are arranged sequentially in order. In Non-Linear DS , elements are arranged hierarchically or interconnected arbitrarily.	Linear: Array, LL, Stack, Queue. Non-Linear: Tree, Graph, Heap.

Algorithm Properties

An algorithm is a step-by-step well-defined computational procedure that takes an input and produces an output.

- **Unambiguous/Clear:** Each step must be clear and distinct.
- **Feasible:** Must execute within available physical resources.
- **Language Independent:** Must be plain logic adaptable to C, C++, Java, Python, etc.
- **Finiteness:** Must terminate after a finite number of steps (no infinite loops).
- **Input/Output:** Must have 0 or more defined inputs and at least 1 well-defined output.

2. Time & Space Complexity Analysis

Algorithm analysis evaluates the efficiency of an algorithm regarding computational time (Time Complexity) and RAM consumption (Space Complexity).

A Priori Analysis:

Theoretical analysis performed *before* code execution. Assumes speed is independent of hardware or processor type. Uses asymptotic notations.

A Posteriori Analysis:

Empirical analysis performed *after* running code on a real machine. Dependent on machine hardware, OS, platform, and compiler choice.

💡 HINGLISH FOCUS (ASYMPTOTIC ANALYSIS):

A Priori Analysis me hum loop counting aur operations calculate karke $O(n)$ nikalte hain. Hum lower order terms (e.g., $n + 10$ ko n batate hain) aur constant multipliers (e.g., $3n/5 + 1$ ko $O(n)$ batate hain) ignore kar dete hain, kyunki high input size ($n \rightarrow \infty$) par highest power rate-determining factor hota hai.

Asymptotic Notations Definition Table

Notation	Mathematical Meaning	Practical Representation
Big Oh — $O(n)$	Upper Bound: $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$	Worst Case Time Complexity (Maximum steps needed).
Big Omega — $\Omega(n)$	Lower Bound: $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$	Best Case Time Complexity (Minimum steps needed).
Theta — $\Theta(n)$	Tight Bound: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	Average / Exact Case Complexity.

🔥 Frequent Exam Question — Hierarchy of Complexity Growth:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Code Analysis & Step Reduction Examples

Example 1 (Nested Loops):

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d", i); // Runs n * n times = n^2
    }
}
// Time Complexity = O(n^2)
```

Example 2 (Polynomial Simplification Rule):

If $T(n) = n^{10} + n + 10$, drop lower-order terms ($n, 10$) $\rightarrow O(n^{10})$.

If loop increments by step 2: $T(n) = n/2 + 1 \rightarrow O(n)$.

If loop executes $3n/5 + 1$ times $\rightarrow O(n)$.

Space Complexity Components

Space Complexity = Fixed Space + Variable/Auxiliary Space

- **Fixed Space:** Program code size, constants, simple scalar variables.
- **Auxiliary Space:** Extra memory dynamically allocated or stack space consumed during recursion.

Example: Iterative vs Recursive Factorial Space Analysis

Implementation Method	Time Complexity	Space Complexity	Reasoning
Iterative Factorial (Loop)	$O(n)$	$O(1)$	Uses constant scalar variables (fact, i).
Recursive Factorial (Call Stack)	$O(n)$	$O(n)$	Requires n recursive activation frames on system call stack.

3. Arrays & Array Operations

An **Array** is a linear collection of homogeneous data elements stored at contiguous memory locations.

💡 HINGLISH CLARIFICATION (ARRAY LIMITATIONS):

Array ki strength hai fast indexing ($O(1)$ time to access `arr[i]` via base address formula) aur

LOCALITY OF REFERENCE

(CPU cache hits maximum hote hain kyunki elements line-by-line hote hain). Par problem do hain:

1.

INTERNAL FRAGMENTATION:

Array fix size 100 ka banaya par use sirf 10 elements kiye.

2.

EXTERNAL FRAGMENTATION:

Memory me total 50 bytes free hain par contiguous (lagatar) free nahi hain, toh bada array allocate nahi ho payega.

Core Operations on Arrays (Algorithms & Code)

A. Insertion at Given Position P

```
// Program to insert element at position P in array
#include <stdio.h>

void insertElement(int arr[], int *n, int pos, int val) {
    // Shift elements to the right from last index down to pos-1
    for (int i = *n; i >= pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos - 1] = val; // Insert value
    (*n)++; // Increment size
}

// Time Complexity: Worst Case =  $O(n)$  [Insert at index 0], Best Case =  $O(1)$  [Insert at end]
```

B. Deletion from Given Position P

```
// Program to delete element at position P
void deleteElement(int arr[], int *n, int pos) {
    // Shift elements left starting from position p-1
    for (int i = pos - 1; i < *n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    (*n)--; // Decrement size
}
// Time Complexity: Worst Case = O(n) [Delete index 0], Best Case = O(1) [Delete last element]
```

4. Linked Lists (Singly, Doubly, Circular)

A **Linked List** is a dynamic linear data structure composed of self-referential structures called *nodes*, connected logically via pointers rather than contiguous memory allocation.

💡 **HINGLISH CONCEPT CLEARER:**

Linked List nodes memory me kahin bhi ho sakte hain. Node ke do part hote hain: Data aur Next Pointer. Iss structures ka fayda ye hai ki insertion/deletion ke waqt shifting nahi karni padti — sirf pointer links update hote hain!

Memory Node Structure in C

```
struct Node {
    int data;
    struct Node *next; // Pointer to next node
};

// Allocation using malloc
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

Array vs Linked List Comparison Table (High-Frequency Exam Question)

Parameter	Array	Linked List
Memory Allocation	Static / Contiguous (Compile-time / Continuous RAM block).	Dynamic / Non-contiguous (Heap allocation via malloc).
Access Time	Random Access O(1) via direct indexing.	Sequential Access O(n) by traversing links.
Insertion / Deletion at Start	Slow O(n) due to right/left element shifting.	Fast O(1) by re-pointing head node.
Insertion / Deletion at End	Fast O(1) (if size known).	O(n) for Singly LL, O(1) if tail pointer maintained.
Memory Overhead	Zero pointer overhead (pure data space).	Extra space per node to store address pointer(s).

Singly Linked List Core Operations Algorithms & C Code

```
/* C Helper Functions for Singly Linked List Deletions */

// Delete from Beginning
void deleteFromBeg(struct Node **head) {
    if (*head == NULL) {
        printf("List Empty\n");
        return;
    }
    struct Node *temp = *head;
    *head = (*head)->next;
    free(temp); // Free dynamic heap memory
}

// Delete from End
void deleteFromEnd(struct Node **head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node *temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

Advanced Linked List Exam Problems

Problem 1: Find N-th Node from the End of a Linked List

Two-Pointer Approach (Slow & Fast Pointers): Move fast pointer *N* steps ahead first. Then move both slow and fast pointers one step at a time until fast reaches NULL. Slow pointer will point to the N-th node from end.

```
struct Node* getNthFromEnd(struct Node* head, int n) {
    struct Node *fast = head, *slow = head;

    // Step 1: Move fast pointer N steps ahead
    for (int i = 0; i < n; i++) {
        if (fast == NULL) return NULL; // Out of bounds
        fast = fast->next;
    }

    // Step 2: Move both together until fast reaches NULL
    while (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow; // Points to N-th node from end
}
```

Problem 2: Merge Two Sorted Linked Lists into One Sorted List

```
struct Node* mergeSortedList(struct Node* head1, struct Node* head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;

    struct Node *head = NULL, *tail = NULL;

    // Initialize head pointer
    if (head1->data < head2->data) {
        head = tail = head1;
        head1 = head1->next;
    } else {
        head = tail = head2;
        head2 = head2->next;
    }

    while (head1 != NULL && head2 != NULL) {
        if (head1->data < head2->data) {
            tail->next = head1;
            tail = head1;
            head1 = head1->next;
        } else {
            tail->next = head2;
            tail = head2;
            head2 = head2->next;
        }
    }

    if (head1 != NULL) tail->next = head1;
    else tail->next = head2;

    return head;
}
```

Doubly Linked List (DLL)

A Doubly Linked List node contains three fields: prev pointer, data, and next pointer. Allows bidirectional traversal.

```
struct DLLNode {
    int data;
    struct DLLNode *prev;
    struct DLLNode *next;
};
```

Advantages of DLL	Disadvantages of DLL	Real-World Applications
• Bidirectional traversal (Forward & Backward) • Deletion of a given node is faster $O(1)$ if node pointer is given.	• Extra memory overhead per node for storing 2 pointers. • More complex pointer manipulation logic during insertion/deletion.	• Web Browser Forward/Back history navigation. • Music Player Previous/Next track list. • Undo/Redo operations in text editors.

Circular Linked List (CLL)

In a Circular Linked List, the last node's next pointer points back to the head node instead of NULL.

- **Types:** Circular Singly Linked List, Circular Doubly Linked List.
- **Applications:** Round-Robin OS process CPU scheduling, Circular Queues, Streaming media playlists.

5. Stacks (LIFO Data Structure)

A **Stack** is an abstract linear data structure operating on the strict **LIFO (Last-In, First-Out)** principle. All insertions (PUSH) and deletions (POP) take place exclusively at a single designated end called the **TOP** of the stack.

💡 HINGLISH KEY TERMINOLOGY:

1.

STACK FRAME:

Function calls ke waqt memory stack me ek block banta hai jisme local variables, arguments aur return address save hota hai.

2.

STACK OVERFLOW:

Jab full stack me PUSH karne ki koshish karein ($TOP == MAXSIZE - 1$).

3.

STACK UNDERFLOW:

Jab khali stack se POP karne ki koshish karein ($TOP == -1$).

Stack Expressions Notation Definitions

Notation Type	Alternative Name	Operator Position	Example
Infix	Standard Algebraic	Operator placed between operands	$A + B$
Prefix	Polish Notation	Operator placed before operands	$+ A B$
Postfix	Reverse Polish Notation	Operator placed after operands	$A B +$

Infix to Postfix Conversion Algorithm & Exam Step Table

Algorithm Summary: Scan Infix expression from Left to Right. If operand, output it immediately. If '(', push to stack. If ')', pop until '('. If operator, pop operators from stack with greater/equal precedence, then push incoming operator.

🔥 Solved Exam Example 1: Convert $Q = A + [(B + C) + (D + E) * F] / G$ to Postfix

Scanned Symbol	Stack Status (Bottom → Top)	Postfix Output Expression String
(	(	Empty
A	(	A
+	(+	A

Scanned Symbol	Stack Status (Bottom → Top)	Postfix Output Expression String
[	(+ [	A
(	(+ [(	A
B	(+ [(	A B
+	(+ [(+	A B
C	(+ [(+	A B C
)	(+ [	A B C +
+	(+ [+	A B C +
(	(+ [+ (	A B C +
D	(+ [+ (	A B C + D
+	(+ [+ (+	A B C + D
E	(+ [+ (+	A B C + D E
)	(+ [+	A B C + D E +
*	(+ [+ *	A B C + D E +
F	(+ [+ *	A B C + D E + F
]	(+	A B C + D E + F * +
/	(+ /	A B C + D E + F * +
G	(+ /	A B C + D E + F * + G
)	Empty Stack	A B C + D E + F * + G / +

🔥 Solved Exam Example 2: Convert $Q = (A + (B * C)^D)$ to Prefix Notation

Prefix Rule:

Reverse Infix String → Apply Infix to Postfix logic (swapping brackets) → Reverse Resulting Output.

1. Reversed Infix: $(D \wedge (C * B) + A)$

2. Postfix of Reversed: $D C B * \wedge A +$

3. Final Reversal to get Prefix:

+ A ^ * B C D

Postfix & Prefix Evaluation Algorithms

Postfix Evaluation Rule (Left to Right): Scan left to right. Push operands to stack. When operator encountered, pop top two operands A (top) and B (next to top). Compute **Result** = $B \text{ ext}\{op\} A$ and push result back to stack.

🔥 Solved Exam Numerical Example: Evaluate Postfix $P = 5, 24, 3, /, +, 2, *$

- Scan 5, 24, 3 → Stack: [5, 24, 3]
- Scan '/' → Pop 3, 24 → Compute $24 / 3 = 8$ → Stack: [5, 8]
- Scan '+' → Pop 8, 5 → Compute $5 + 8 = 13$ → Stack: [13]
- Scan 2 → Stack: [13, 2]
- Scan '*' → Pop 2, 13 → Compute $13 * 2 = 26$ → Final Value =

26

Complete C Program: Dynamic Stack Implementation (Using Linked List)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
} *top = NULL;

void push(int val) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow (Heap Full)\n");
        return;
    }
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    struct Node *temp = top;
    printf("Popped element: %d\n", top->data);
    top = top->next;
    free(temp);
}

void display() {
    struct Node *temp = top;
    printf("Stack: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

■ 6. Queues (FIFO Data Structure)

A **Queue** is a linear dynamic data structure following the **FIFO (First-In, First-Out)** principle. Operations occur at two distinct ends: Insertions happen at the **REAR** end (Enqueue), and Deletions happen at the **FRONT** end (Dequeue).

💡 HINGLISH EXPLANATION (WHY CIRCULAR QUEUE?):

Simple Linear Queue me ek badi problem hai: Jab hum kuch elements delete karte hain, toh FRONT pointer aage khisak jata hai. Par Jab $REAR == MAXSIZE - 1$ ho jata hai, tab aage space hote hue bhi hum naye element insert nahi kar pate ("False Overflow"). Iss memory wastage ko hatane ke liye

CIRCULAR QUEUE

me array ke end ko start se connect kar dete hain using Modulo Arithmetic: $(index + 1) \% MAXSIZE$.

Queue Conditions Summary Table

Queue Type	Underflow Condition	Overflow Condition	Pointer Increments
Linear Queue (Array)	$ext\{FRONT\} == -1$ or $ext\{FRONT\} > ext\{REAR\}$	$ext\{REAR\} == ext\{MAXSIZE\} - 1$	rear++; front++;
Circular Queue	$ext\{FRONT\} == -1$	$(ext\{REAR\} + 1) \% ext\{MAXSIZE\} == ext\{FRONT\}$	rear = (rear + 1) % MAX; front = (front + 1) % MAX;

Circular Queue Standard Algorithms

Algorithm 1: Enqueue (Insert in Circular Queue)

```
Algorithm CircularEnqueue(Q, MAXSIZE, FRONT, REAR, ELEMENT)
1. IF (REAR + 1) % MAXSIZE == FRONT THEN
    Print "Queue Overflow" and Return
2. IF FRONT == -1 THEN // First element insertion
    Set FRONT = 0, REAR = 0
    ELSE
    Set REAR = (REAR + 1) % MAXSIZE
3. Set Q[REAR] = ELEMENT
4. EXIT
```

Algorithm 2: Dequeue (Delete from Circular Queue)

```
Algorithm CircularDequeue(Q, MAXSIZE, FRONT, REAR)
1. IF FRONT == -1 THEN
    Print "Queue Underflow" and Return
2. Set ELEMENT = Q[FRONT]
3. Print "Deleted Element = ", ELEMENT
4. IF FRONT == REAR THEN // Queue becomes empty after deletion
    Set FRONT = -1, REAR = -1
    ELSE
    Set FRONT = (FRONT + 1) % MAXSIZE
5. EXIT
```

🔥 Solved Dry Run Exam Problem: Circular Queue Execution Trace

Given Array Size = 6. Perform Enqueue 10, 20, 30, 40, Dequeue 2 elements, Enqueue 50, 60, 70.

- Enqueue 10, 20, 30, 40 → **Front = 0, Rear = 3**, Queue = [10, 20, 30, 40, _, _]
- Dequeue 2 items (10, 20) → **Front = 2, Rear = 3**, Queue = [_, _, 30, 40, _, _]

3. Enqueue 50, 60 → $Rear = 5$, Queue = [_, _, 30, 40, 50, 60]

4. Enqueue 70 → $Rear = (5+1)\%6 = 0$, Queue = [70, _, 30, 40, 50, 60] (*Rear wraps around to 0!*)

7. Exam Rapid Revision Cheat Sheet

Data Structure	Access / Search	Insertion	Deletion	Primary Application
Array	$O(1)$ / $O(n)$	$O(n)$	$O(n)$	Fast lookup tables, linear data buffers.
Singly Linked List	$O(n)$ / $O(n)$	$O(1)$ at head	$O(1)$ at head	Dynamic memory allocation, polynomial representation.
Doubly Linked List	$O(n)$ / $O(n)$	$O(1)$ at head/ tail	$O(1)$ given node ptr	LRU Cache implementation, navigation systems.
Stack	$O(n)$ / $O(n)$	$O(1)$ (Push)	$O(1)$ (Pop)	Recursion, Expression parsing, Backtracking.
Queue	$O(n)$ / $O(n)$	$O(1)$ (Enqueue)	$O(1)$ (Dequeue)	CPU scheduling, Printer spools, BFS traversal.